Auto Typer — User and Developer Documentation

Version: Based on the supplied Python source
Application type: Desktop GUI
Primary purpose: Automatically types prepared text into the currently focused application, with configurable speed and optional simulated typing mistakes.

1. Overview

Auto Typer is a Python desktop application built with Tkinter. It lets a user enter text, choose a typing speed, and send that text as keyboard input to another window. Optional features include simulated mistakes that are corrected with backspaces, a start delay, a progress indicator, and an optional mouse click at a saved screen position.

The application uses `pynput` to control the keyboard and mouse. It can optionally read Microsoft Word `.docx` files using `python-docx`.

The source describes the tool as operating without an API key and without an internet connection after required dependencies have been installed. On first launch, it may attempt to install missing Python packages through `pip`, which does require package access.

2. Features

- **Text editor:** Type or paste text directly into the application.
- **Clipboard paste:** Replace the editor contents with the current clipboard text.
- **Word document import:** Load `.docx` paragraphs and tables into the editor.
- **Typing-speed control:** Set a speed from 10 to 300 words per minute (WPM), with presets:
  - Slow — 35 WPM
  - Normal — 65 WPM
  - Fast — 110 WPM
  - Turbo — 180 WPM
- **Simulated typing mistakes:** Optionally enter a neighboring QWERTY letter, pause, backspace over the mistake and any following characters, and retype the correct text.
- **Mistake rate:** Configure a displayed rate from 0% to 30%.
- **Start delay:** Choose a delay from 1 to 60 seconds; the default is 5 seconds.
- **Always-on-top window:** Keep the Auto Typer window above other windows.
- **Optional mouse click:** Save a screen coordinate and have the application move the mouse there and click before typing.
- **Progress and status:** View character and word counts, estimated duration, progress, and current status.
- **Emergency stop:** Press `Esc` while typing to request a stop.

- **Cross-platform font selection:** The UI chooses a default font based on Windows, macOS, or other platforms. Actual keyboard-control behavior can vary by operating system and application permissions.

3. Requirements

- Python 3.
- `tkinter`, usually included with standard Python distributions; on some Linux distributions it may need to be installed separately.
- `pynput`, required for keyboard and mouse control.
- `python-docx`, optional and used only for importing Word `.docx` documents.

The program attempts to install `pynput` at startup if it is missing. It attempts to install `python-docx` when the user imports a Word document and the package is missing. Automatic installation can fail because of network, permissions, Python environment, or package-manager issues.

4. Installation and launch

1. Install Python 3 from the official Python distribution for your operating system.
2. Save `auto_typer.py` to a local folder.
3. Run the script from a terminal:

   ```
   python auto_typer.py
   ```

4. If the keyboard-control dependency cannot be installed automatically, install it manually:

   ```
   python -m pip install pynput
   ```

5. To enable Word document import, install:

   ```
   python -m pip install python-docx
   ```

The supplied source mentions launching through a `run_windows.bat` file, but that batch file was not included in the reviewed source, so its exact behavior is not documented here.

5. Quick start

1. Launch Auto Typer.
2. Enter text in the **Text to type** editor, or use **Paste** or **Open Word doc**.
3. Choose a speed using the slider or one of the presets.
4. Choose whether simulated mistakes should be enabled and set the mistake rate if desired.
5. Set the start delay.
6. If you want the application to click a target location automatically, enable the mouse-click option and use **Set click spot**. During its three-second countdown, move the mouse to the target location.

7. Press **Start typing**.
8. During the countdown, click the destination application and position the text cursor where the text should appear, unless the saved mouse-click option will do that for you.
9. Press `Esc` or click **Stop (Esc)** to stop the run.

Important: Test with a small, non-sensitive text sample first. Make sure the destination window and insertion point are correct before starting.

6. User interface reference

Text to type

The main editor accepts manually entered text. **Paste** replaces the editor content with clipboard text. **Clear** empties the editor. **Open Word doc** opens a file picker for `.docx` files.

The importer reads document paragraphs and table rows. Table cells are joined with tab characters; paragraph and table content is placed into a sequence of lines. Complex Word formatting, images, headers, footers, and other document features are not reproduced as formatted content.

Typing speed

The slider ranges from 10 to 300 WPM. Preset buttons set common values. The internal delay is based on the conventional approximation of five characters per word, with random timing variation. Actual observed speed may differ because of punctuation, simulated mistakes, system load, and target application behavior.

Add typing mistakes

When enabled, the app may replace an alphabetic character with a neighboring key on a QWERTY layout. It then pauses, uses backspace to remove the incorrect sequence, and types the intended sequence again. This is a simulation feature, not a spelling or grammar checker. Characters without a defined neighboring-key mapping may not receive a simulated error.

The rate slider is displayed from 0% to 30%. Because the rate is applied probabilistically to eligible alphabetic characters, the actual number of mistakes varies from run to run.

Start delay

The delay gives the user time to focus the destination window. The control accepts 1–60 seconds, defaulting to 5 seconds. Invalid values fall back to 5 seconds.

Keep this window on top

This setting controls whether the Auto Typer window stays above other windows.

Click a spot before typing

When enabled, Auto Typer moves the mouse toward the saved screen coordinate and performs a left click before typing. Use **Set click spot (3 second countdown)** to capture the current mouse position after the countdown. The option requires a saved position.

The click position is a screen coordinate, not a named application or control. Screen layout changes, display scaling, or moving windows may make the saved coordinate point to the wrong place.

Progress and status

The application shows character and word counts, an approximate time estimate, a progress bar, and a status message. The estimate is a rough calculation and is not a guarantee of completion time.

7. Keyboard controls

Control	Behavior
`Esc`	Requests a stop while typing is active.
Start typing	Begins the countdown and typing run.
Stop (Esc)	Requests a stop for the active run.

The stop mechanism is cooperative: the worker checks the stop event between characters and during timed pauses. A key event or operation already in progress may complete before the stop takes effect.

8. Technical architecture

Main components

- **`load_pynput()`** — Imports keyboard and mouse controllers, listener, and button definitions; attempts to install `pynput` if unavailable.
- **`load_docx()`** — Imports `python-docx`, attempting installation if necessary.
- **`FlatButton`** — A label-based clickable widget with hover styling.
- **`Toggle`** — A custom canvas-based on/off switch.
- **`AutoTyperApp`** — Owns the UI, configuration, keyboard/mouse controllers, typing thread, and application lifecycle.
- **`main()`** — Creates the Tkinter root, loads dependencies, displays an error if keyboard control cannot be initialized, and starts the GUI event loop.

Typing workflow

1. Validate that the editor contains non-whitespace text.
2. If mouse-click mode is enabled, validate that a target coordinate is saved.
3. Set the run state and start a background worker thread.
4. Count down for the configured delay, checking for a stop request.
5. Optionally move the mouse to the saved position and click.

6. Process the text one character at a time.
7. Apply randomized per-character delays and optional mistake/correction sequences.
8. Update the status and progress display.
9. Mark the run complete or stopped.

Threading and UI updates

Typing runs on a daemon thread so the UI can remain responsive. Status and progress updates are scheduled onto Tkinter's event loop using `root.after()`. A threading event is used to coordinate stopping.

9. Configuration defaults

Setting	Default	Range / choices
Typing speed	65 WPM	10–300 WPM
Simulated mistakes	Enabled	On / Off
Mistake rate	6%	0–30%
Start delay	5 seconds	1–60 seconds
Always on top	Enabled	On / Off
Mouse click before typing	Disabled	On / Off; requires saved position

10. Troubleshooting

The application does not start - Confirm Python 3 is installed. - Run the script from a terminal to inspect any error messages. - Check that Tkinter is available in your Python installation.

Keyboard or mouse control does not work - Install or upgrade `pynput` in the same Python environment used to launch the app. - Check operating-system accessibility/input-monitoring permissions where applicable. - Some protected or elevated applications may not accept simulated input from a process running with different permissions.

Word import fails - Install `python-docx` with `python -m pip install python-docx`. - Confirm the file is a valid `.docx` document rather than an older `.doc` file. - If the document is complex, expect only paragraph and table text to be imported.

Text appears in the wrong place - Stop the run immediately. - Disable mouse-click mode or set the click coordinate again. - Verify the intended destination window and text cursor before starting another run.

Typing is slower than expected - Lower the mistake rate or disable simulated mistakes. - Remember that the WPM setting is approximate and punctuation introduces extra pauses. - Try a short sample to determine a suitable speed for the target application.

11. Limitations and safety notes

- The program sends keyboard and mouse input to the active desktop; it does not target a specific application by name.
- It does not validate the destination field or confirm that the text was accepted.
- The Word importer does not preserve document formatting.
- Speed and completion-time figures are estimates.
- The simulated mistake behavior is intended to imitate correction pauses; it does not guarantee human-like typing.
- Automated input may violate the rules of some websites, assessments, games, or workplace systems. Use it only where permitted.
- Avoid using the tool in password fields, payment forms, or other sensitive contexts.
- Review the destination content after typing, especially when formatting or line breaks matter.

12. Developer notes and maintenance

- Keep dependency installation failures visible during development; the current installer suppresses `pip` output, which can make diagnosis harder.
- Consider pinning tested dependency versions in a `requirements.txt` file for reproducible installations.
- A separate launcher such as `run_windows.bat` should be documented alongside the script if distributed.
- Test keyboard input, stop behavior, document import, and mouse-coordinate handling on each supported operating system before claiming full cross-platform compatibility.

13. License and support

The supplied source does not specify a license, author contact, or support channel. Add those details before publishing or distributing the application.